

SAP-Assignment 2

Shipping on the Air

Marco Morelli

1. QUALITY ATTRIBUTES

Il sistema descritto in questa relazione è un prototipo di consegna pacchi tramite droni, evoluzione dell'Assignment 01, in cui il design, l'implementazione e il deployment vengono raffinati applicando un sottoinsieme di microservices pattern. Prima di descrivere i pattern vengono individuati due **Quality Attribute (QA)** rappresentativi, di cui si specifica lo scenario: essi costituiscono le due proprietà che i pattern di observability rendono osservabili e verificabili, e sono verificati **end-to-end** secondo quanto descritto nella Sezione 4.

Il primo QA riguarda la **responsiveness** e ne fissa il requisito minimo in condizioni di overload.

QA Scenario: Responsiveness

Quality attribute scenario : Responsiveness

Feature : Small average response time in case of overload
when initiate 1000 concurrent requests
caused by 1000 users
occur in the operating
system in normal operation
then the system processes all requests
so that the average response time is < 100 milliseconds

Il secondo QA riguarda la **gestione dei partial failure**: quando l'account service diventa irraggiungibile, il sistema deve evitare di propagare richieste destinate a fallire.

QA Scenario: Handling partial failures

Quality attribute scenario : Handling partial failures

Feature : Avoid pointless requests in case of unavailability of the account service
when more than 50% of total requests fail
caused by unavailability of the account service
occur in the operating
system in normal operation
then the system makes the requests fail immediately
without propagating to the account service
so that it lightens the workload avoiding pointless requests

I due scenari orientano le scelte architetturali: la responsiveness motiva l'adozione di un modello di I/O non bloccante del gateway e la misura del tempo di risposta a livello di edge come il pattern stesso indica fra le proprie design issue; la gestione dei partial failure motiva il **Circuit Breaker** sull'account proxy del gateway. In entrambi i casi un pattern realizza la proprietà e una metrica **Prometheus** la rende verificabile.

2 DEPLOYMENT

Ogni microservizio è deployato in un singolo container Docker. L'intero sistema è orchestrato con **Docker Compose**, che consente di avviarlo con un solo comando (*docker compose up --build*) a partire dalle immagini buildate direttamente dai rispettivi Dockerfile. Tutti i servizi condividono un'unica rete (*shipping_network*): questa rete condivisa permette di abilitare il **Service Discovery via DNS** e lo scrape di **Prometheus**.

I **container** usano base image *eclipse-temurin-25* e vengono avviati come *fat-jar*. Tale scelta è preferibile all'avvio tramite *mvn exec:java*, che eseguirebbe Maven a runtime rendendo l'avvio più lento e meno prevedibile: un avvio rapido e deterministico è importante perché il container è sottoposto a *healthcheck* e alla supervisione *dell'autoheal*, come discusso nella Sezione 3.2.1.

I due servizi che detengono stato di dominio (*account-service* e *delivery-service*) sono persistiti su named volume dedicati (*account-data* e *delivery-data*), montati sulla directory *data/* che contiene il rispettivo store su file. Le credenziali degli account e la storia degli eventi delle consegne sopravvivono quindi alla ricreazione dei container. I seeder restano idempotenti: *AdminSeeder* risemina l'account amministratore solo se non è già presente nello store persistito, mentre *FleetSeeder* risemina la flotta a ogni avvio, poiché il fleet è un bounded context in-memory che non è event-sourced.

È la persistenza coerente dei due servizi a rendere le consegne effettivamente utilizzabili dopo un riavvio. Una consegna è associata al proprio mittente attraverso il *senderId*, e finché sopravvivono entrambi (la consegna e l'account che la possiede) il mittente le ritrova rieffettuando il login. Diverso è il caso delle sessioni, mantenute in-memory dal *session-service* e non persistite: al riavvio l'utente ne ottiene una nuova, ma con lo stesso *accountId*, sufficiente a riconoscere la titolarità delle consegne. A persistere è dunque lo stato di dominio, non l'identità di sessione, volatile per costruzione.

Questa configurazione preserva entrambi i benefici che la teoria attribuisce all'Event Sourcing: la strutturazione della business logic attorno a eventi con ricostruzione dello stato per replay, e la conservazione durevole della storia dell'aggregate. Il primo è realizzato dall'aggregate Delivery, ricostruito rigiocando i propri eventi; il secondo dalla persistenza dello store su volume. La durabilità dello store abilita il recovery al riavvio descritto nella Sezione 3.3.

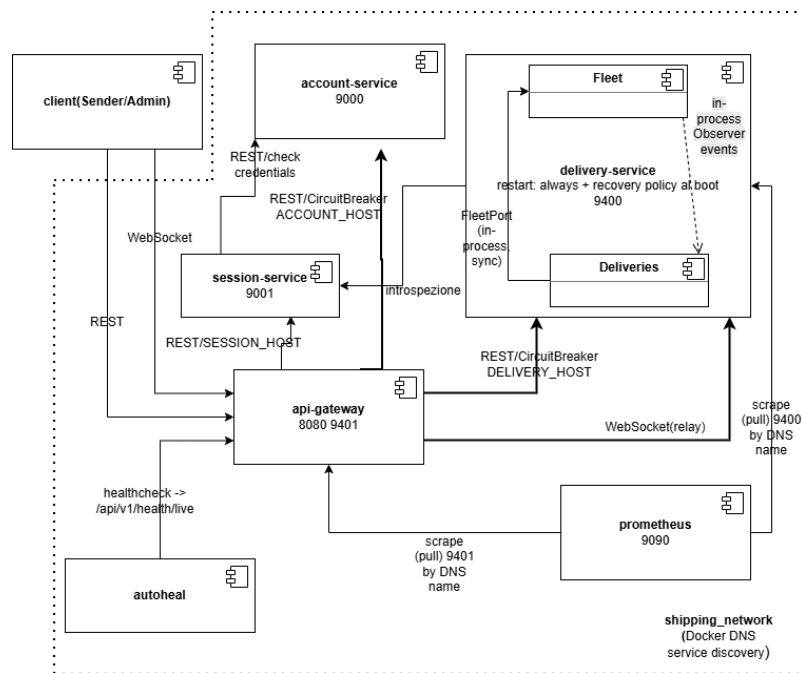
Due comandi **Compose** riflettono i due usi dei volumi: *docker compose down* arresta il sistema conservando account e consegne, che alla ripartenza vengono ricaricati; *docker compose down -v* rimuove i volumi e riporta il sistema a uno stato pulito. Un override esplicito per servizio (*ACCOUNT_RESET_STORE*, *DELIVERY_RESET_STORE*, default disattivi) consente di forzare la ripartenza pulita del singolo store senza rimuovere i volumi.

Tutti i servizi applicativi adottano *restart: always*. Per account-service, session-service e api-gateway il riavvio è sicuro perché il loro stato è recuperabile all'avvio: l'account ricarica le credenziali dal volume persistito (con l'amministratore riseminato in modo idempotente se assente), la sessione riparte pulita in-memory, il gateway è privo di stato di dominio.

Per il delivery-service il riavvio automatico è reso sicuro dalla stessa condizione: al boot lo stato è ricostruibile in modo coerente, poiché le consegne sono ricostruite dall'event store persistito mentre la flotta riparte pulita dal seeder. Il riallineamento fra i due piani, consegne persistenti e flotta effimera, è realizzato dalla recovery policy descritta nella Sezione 3.3. Il riavvio è quindi sicuro perché lo stato risultante è sempre coerente, non perché lo stato precedente sia integralmente ripristinato.

3 PATTERNS

Rispetto all'Assignment 1 sono stati applicati i seguenti pattern: **API Gateway, Health Check API, Application Metrics, Event Sourcing, Service Discovery e Circuit Breaker**. L'architettura rimane esagonale con *DDD*: ogni servizio è un progetto Maven autonomo e self-contained. Il sistema finale è composto da quattro microservizi deployabili: *account-service*, *session-service*, *delivery-service* e *api-gateway*.



Deployment di sistema

3.1 API Gateway

È stato introdotto un microservizio **api-gateway** come unico entrypoint del sistema, con il compito di nascondere la composizione interna dei servizi. Esso espone la **REST API** lato client e, tramite l'*APIGatewayController*, instrada ogni richiesta al servizio a valle sfruttando una routing map e i relativi proxy (*AccountServiceProxy*, *SessionServiceProxy*, *DeliveryServiceProxy*).

Il gateway è un router puro: non contiene logica di dominio, non compone risposte di più servizi né coordina transazioni distribuite. Ogni comando si risolve nell'invocazione di un singolo servizio a valle secondo la routing map. Il gateway inoltra le credenziali al session-service e propaga ai servizi a valle l'identificativo di sessione tramite l'header *X-Session-Id*, senza derivare né iniettare l'identità del chiamante nel payload.

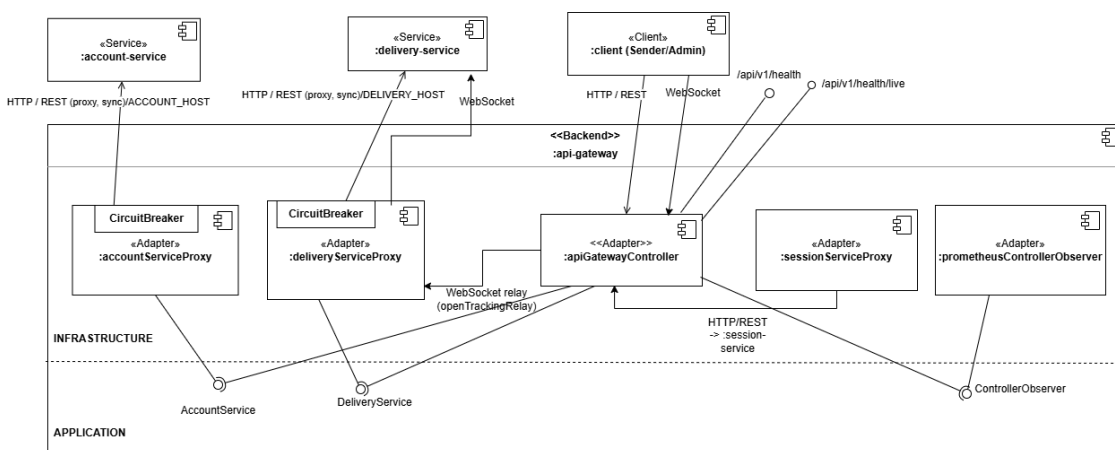
La gestione dell'identità evolve la soluzione dell'Assignment 1. Il session-service resta un bounded context separato: autentica al login, custodisce la sessione ed espone l'introspezione dell'identità ai servizi interni. L'autorizzazione per caso d'uso è collocata nel delivery-service, che risolve l'identità interrogando il session-service e resta sicura anche se invocata direttamente sulla propria porta. Il gateway conserva così le sole responsabilità che il pattern gli attribuisce (routing e protocol translation) mentre autenticazione e autorizzazione restano nei servizi che ne sono proprietari.

Il meccanismo di propagazione dell'identità è "introspezione", ereditata dall'Assignment 1: a ogni richiesta autorizzata il delivery-service interroga il session-service per ottenere *accountId* e ruolo alla fonte. Il trade-off, che si documenta, è un hop di rete aggiuntivo per ogni richiesta autorizzata; è un costo dichiarato, mitigabile in evoluzione futura adottando un token (JWT).

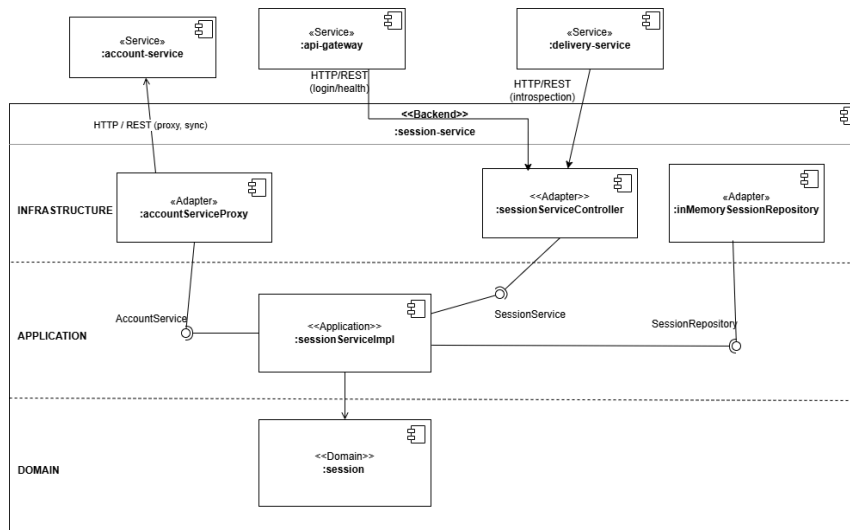
Il tracking real-time, realizzato nell'Assignment 1 con una **WebSocket** diretta client-delivery, diventa un relay in due tratte: il gateway accetta la WebSocket dal client e apre una WebSocket interna verso il delivery, facendo da relay dei messaggi di posizione ed ETR. Ciò rientra nella protocol translation attribuita al gateway per la mediazione fra protocolli client-friendly e protocolli interni. Anche qui si dichiara un trade-off: il gateway mantiene una connessione aperta per ciascun utente che effettua il tracking.

3.1.1 Session-service

Il session-service è un dominio a tutti gli effetti, classificato come **generic subdomain** poiché non è il core domain, che resta Deliveries. Il suo aggregate *Session* è dotato di identità, ciclo di vita ed evento *SessionCreated*. Autentica le credenziali delegandone la verifica all'account-service e materializza l'esito in una sessione. Mantiene le sole regole del proprio linguaggio, mentre l'autorizzazione per caso d'uso appartiene al linguaggio ubiquo di Deliveries ed è applicata dal delivery-service.



C&C dell'api-gateway



C&C del session-service

3.2 Observability Patterns

3.2.1 Health Check API

Ogni servizio espone un endpoint di health check. Sul gateway si sono distinti **liveness** e **readiness**:

- *GET /api/v1/health/live (liveness)*: risponde sempre 200/UP senza interrogare i downstream, a indicare che il processo del gateway è attivo.
- *GET /api/v1/health (readiness aggregato)*: sonda account, session e delivery e risponde 200/UP se entrambi sono disponibili, 503/DOWN altrimenti.

Questa distinzione ha una conseguenza operativa precisa: l'*healthcheck* di **Docker Compose** è configurato sulla liveness, cosicché l'*autoheal* riavvia il gateway solo se il processo è effettivamente terminato e non, ad esempio, durante un avvio ancora in corso o per l'indisponibilità temporanea di un downstream. Con un unico health aggregato, l'indisponibilità di un downstream avrebbe reso il gateway unhealthy, inducendo l'*autoheal* a riavviarlo inutilmente e mascherando così il vero guasto. Il readiness resta invece utile per la diagnosi e per la chiusura del Circuit Breaker.

3.2.2 Application Metrics

Si utilizza **Prometheus**, aggiunto al **Compose** e configurato tramite *prometheus.yml*. Due servizi espongono metriche, ciascuno su una porta dedicata indicata per **DNS name**.

Il Service Discovery abilita lo scrape:

delivery-service

metriche di livello applicativo (porta 9400):

- *created_deliveries*: numero totale di consegne create.
- *deliveries_in_progress*: consegne attualmente in corso.
- *deliveries_delivered*: numero totale di consegne completate.

api-gateway

metriche di livello infrastrutturale (porta 9401):

- *rest_requests*: numero totale di richieste REST.
- *request_response_time_seconds*: tempo di risposta cumulativo in secondi (QA responsiveness).
- *account_circuit_open*: stato del Circuit Breaker dell'account (QA partial failures).

Coerentemente con l'architettura esagonale, la metrica risiede **al di fuori della business logic**: il delivery-service impiega l'adapter *PrometheusDeliveryServiceObserver* per esporre le metriche implementando l'interfaccia *DeliveryServiceEventObserver*, mentre il gateway impiega *PrometheusControllerObserver* per le metriche implementando *ControllerObserver*, osservato dall'*APIGatewayController*.

3.3 Event Sourcing

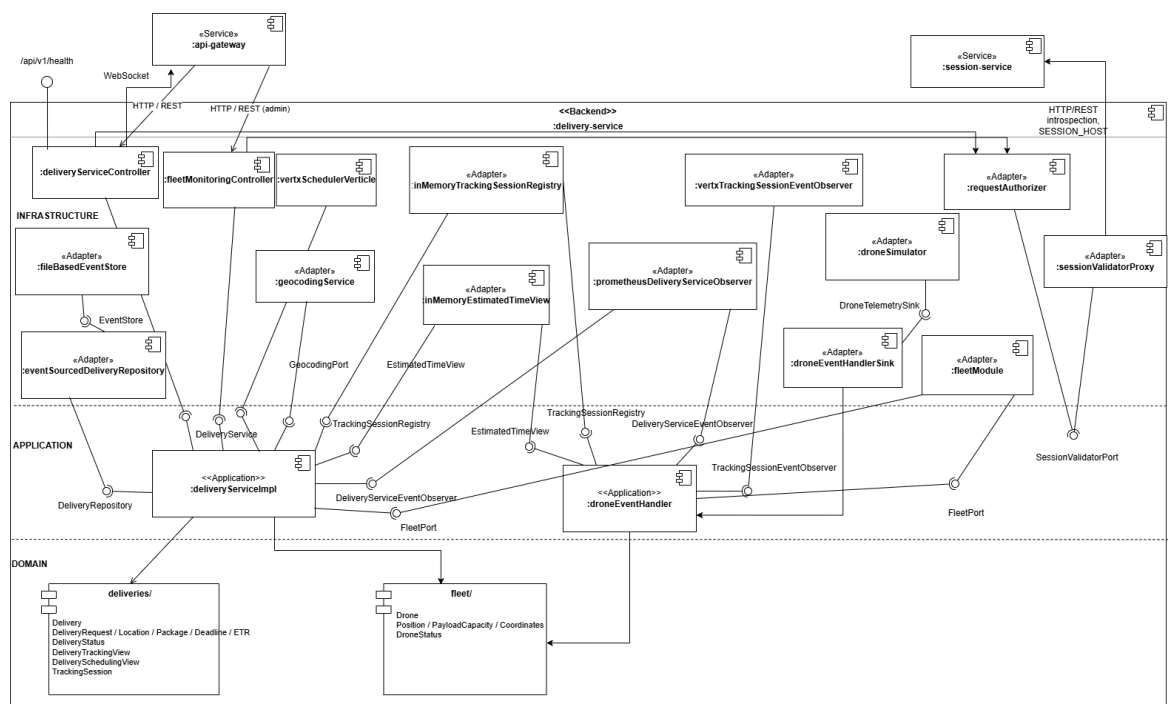
L'Event Sourcing è applicato al **delivery-service**: è il servizio che meglio si presta a essere strutturato a eventi, poiché il ciclo di vita di una consegna è già una sequenza di transizioni di stato per le quali erano stati definiti domain event. L'aggregate *Delivery* è persistito come **sequenza di eventi**, e non come stato: una porta applicativa *EventStore* con adapter *FileBasedEventStore*, e un *EventSourcedDeliveryRepository* che effettua l'append degli eventi e ricostruisce l'aggregate tramite replay.

Ogni transizione di stato emette un evento e ogni tipo di evento dispone del proprio ramo in *apply()*, cosicché il replay non può mai fallire. Un solo evento, *EstimatedTimeUpdated*, è escluso dalla persistenza pur essendo emesso sull'aggregate: rappresenta il tempo stimato residuo, che varia a ogni tick del volo e costituisce un dato di **read-model volatile**.

L'*EventSourcedDeliveryRepository* lo filtra esplicitamente in fase di append, cosicché non gonfi lo store senza apportare valore storico. La posizione del drone, *PositionUpdated*, ricade nella stessa categoria di stato volatile ma per una ragione strutturalmente diversa: appartiene al bounded context fleet, gestito da un repository in-memory, e non entra affatto nel perimetro dell'event store. Emessa a ogni tick dal Drone, viene consumata in-process

per calcolare l'ETR, aggiornare il read-model e alimentare il relay di tracking, ma non è mai persistita. Si mantengono così distinti due piani: la **persistenza a eventi** (event store, lato write) e la **notifica a eventi** (Observer in-process per tracking e metriche).

Lo **store** è persistito su volume, e questa durabilità abilita un recovery al riavvio del servizio. Al boot, dopo il seeding della flotta e prima dell'apertura del traffico, un *DeliveryRecoveryService* ricostruisce le consegne dall'event store e ne riallinea lo stato con la flotta, che riparte pulita. La policy distingue tre casi: le consegne già completate o annullate non vengono toccate; le consegne immediate in corso vengono riavviate da capo, riassegnando un drone dalla flotta e facendo ripartire il volo dall'origine, poiché la posizione è un dato volatile non persistito; le consegne programmate ancora prenotate vengono annullate, rilasciando la prenotazione e riportando il drone a disponibile. Il recovery è idempotente: un secondo riavvio non altera lo stato già riallineato. In questo modo l'event store è la sola sorgente di verità persistita, e lo stato del fleet resta una proiezione derivata, ricostruita a ogni avvio.



C&C del delivery-service

3.4 Service Discovery

Il Service Discovery è realizzato a **livello di deployment tramite Docker**: la piattaforma dispone di un service registry integrato e di un *DNS resolver* interno, e assegna a ogni servizio un DNS name. I proxy del gateway puntano ai nomi logici (*account-service*, *delivery-service*, *session-service*),

letti da variabili d'ambiente

(*ACCOUNT_HOST/DELIVERY_HOST/SESSION_HOST*) con default localhost, e non a indirizzi IP.

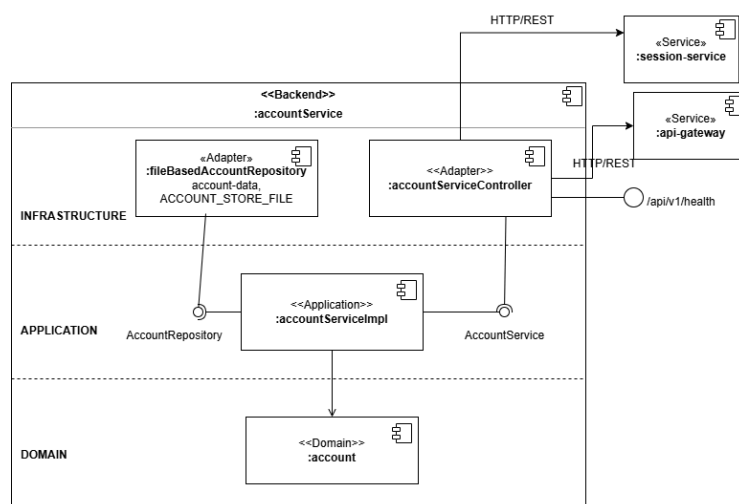
3.5 Circuit Breaker

Il **Circuit Breaker** è applicato all'account proxy del gateway in modo da risolvere parzialmente il problema legato alla possibilità di **partial failure** durante la comunicazione sincrona con altri servizi.

Il **CircuitBreaker** modella tre stati *CLOSED* / *OPEN* / *HALF_OPEN*. Esso traccia i successi e i fallimenti su una finestra con volume minimo: superata la soglia del **50%** di fallimenti, il circuito si apre e il gateway risponde immediatamente con errore senza inoltrare la richiesta. Trascorso un **timeout di 10 secondi**, il breaker passa in *HALF_OPEN* e sonda l'*health* del servizio: se questo è tornato disponibile, il circuito si richiude.

L'aspetto architetturalmente più delicato è **che cosa costituisce un fallimento**. Il breaker conta esclusivamente i guasti del downstream: errori di trasporto o timeout del WebClient e risposte 5xx. Non concorrono invece le risposte applicative 4xx, che indicano che il servizio ha risposto correttamente rifiutando l'input.

Il breaker alimenta la metrica *account_circuit_open* (tramite callback di cambio stato), rendendo osservabile il QA sui partial failure. Il relay WebSocket non attraversa il breaker, trattandosi di uno stream e non di un'interazione request/response sincrona.



C&C dell'account-service

4 TESTING

La strategia di test copre l'intera **test pyramid**. Unit, integration e component test risiedono nel *src/test* di ciascun servizio. Gli end-to-end test risiedono in un modulo separato di soli test (*system-end-to-end*). In ogni servizio sono inoltre presenti test architetturali ArchUnit, che verificano il rispetto dell'architettura esagonale.

4.1 Unit testing

Gli **unit test** seguono la distinzione *solitary/sociable* in base al ruolo della classe. Nel *delivery-service*, ad esempio, il dominio è testato in modalità **sociable** (aggregate Delivery, value object quali peso, coordinate, deadline ed ETR, aggregate Drone), poiché la logica è distribuita tra più oggetti mentre l'application (*DeliveryServiceImpl*) è testata in modalità **solitary**, con le porte mockate.

4.2 Integration testing

Gli **integration test** verificano l'interazione con i sistemi esterni senza avviare l'intera applicazione. Un esempio rappresentativo è il *DeliveryServiceProxy* del gateway verificato contro un delivery service simulato, che ne valida il contratto **REST** verso il servizio a valle. A questo livello si collocano anche i test dell'event store su file, dello scheduler a timer reale, del relay WebSocket, dell'health aggregato e dell'esposizione delle metriche **Prometheus**. A livello di test è inoltre presente la verifica della recovery policy del delivery-service.

4.3 Component testing

I **component test** verificano un singolo servizio come *black box*, attraverso la sua API, mediante **Cucumber/Gherkin**:

- **account-service**: registrazione, login.
- **delivery-service**: creazione immediata e programmata, rifiuti di dominio, avvio del tracking, viste read-only dell'amministratore su flotta e scheduling, controlli di accesso diretto e autorizzazione.

4.4 End-to-end testing

Gli **end-to-end test** sono richieste esterne verso il sistema completo, effettuate dopo *docker compose up*, secondo l'approccio **user journey**, anch'essi redatti in Gherkin ed eseguiti via Cucumber. Sono definiti tre journey:

- **Creazione delivery:** registrazione → login → creazione immediata → recupero del dettaglio. Esercita la catena gateway → account → delivery.
- **Tracciamento delivery:** (registrazione, login e creazione come precondizioni) → avvio del tracking via relay WebSocket (ricezione di frame reali) → lettura dello status → arresto. Verifica il relay client-gateway-delivery.
- **Cancellazione delivery:** registrazione → login → creazione schedulata → cancellazione delivery.

Nello stesso modulo risiedono i due test che verificano i **QA** scenario della Sezione 1, ed è in questo punto che observability e pattern si compongono in un'unica storia:

- **PerformanceTest (responsiveness):** invia 1000 richieste concorrenti al gateway e calcola il tempo medio dal *delta* delle metriche, verificando che resti al di sotto di 100 ms. Il probe interroga la liveness, così da misurare la latenza del solo gateway senza falsi 503 sotto carico.
- **CircuitBreakerTest (partial failures):** disconnette effettivamente l'account dalla rete Docker, osserva la gauge *account_circuit_open* passare a 1, riconnette il servizio e verifica il ritorno a 0. Il gradino 0→1→0 costituisce la prova visiva del pattern.

5 CONCLUSIONI

L'adozione di Docker e Docker Compose ha reso il sistema portabile, riproducibile ed estendibile con un solo comando di avvio; il container per servizio garantisce isolamento e un minore accoppiamento a runtime.

L'applicazione dei pattern ha migliorato il sistema su assi distinti:

- L'API Gateway incapsula la struttura interna dietro un unico endpoint come router puro, mentre autenticazione e autorizzazione restano distribuite fra il session-service e i servizi proprietari delle rispettive invarianti.
- Gli observability pattern abilitano il monitoraggio, la diagnosi e l'alerting.
- L'Event Sourcing struttura la persistenza della consegna come storia di eventi durevole, che abilita la ricostruzione dello stato e il recovery coerente al riavvio.
- Service Discovery e Circuit Breaker accrescono la robustezza del sistema rispetto alla sua natura distribuita.

Il valore architetturale non risiede unicamente nell'aver introdotto i pattern, bensì nell'averli integrati in modo coerente: i due Quality Attribute sono realizzati da un pattern e resi verificabili da una metrica, e la copertura di test lungo l'intera pyramid, culminata negli user journey e nei due test di QA end-to-end, collega la correttezza delle singole funzionalità a quella delle comunicazioni tra i servizi e del sistema nel suo complesso.